

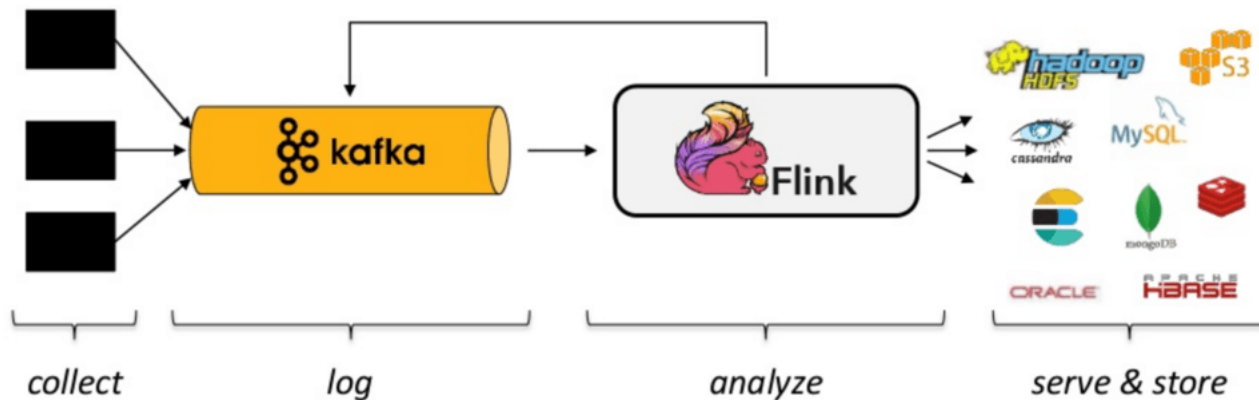
Stream, Dataflow programming, and Distributed Snapshot



CUHK

Streaming systems

- https://youtu.be/F8I9suOJ_DM
 - Courtesy: Chris Liu



Community > Blog > Four Billion Records per Second! What is Behind Alibaba Double 11 - Flink Stream-Batch Unification Practice during D

Four Billion Records per Second! What is Behind Alibaba Double 11 - Flink Stream-Batch Unification Practice during Double 11 for the Very First Time

Alibaba Clouder

December 2, 2020

 2,395

 0

Flink

- Programming Model
 - Stream as first class citizen
 - Batch is just a bounded stream
 - Dataflow programming model
- Exactly-once Consistency (~~At least once~~)
 - Each incoming event affects the final results exactly once
 - Even in case of a machine or software failure
 - there's no duplicate data due to resend and no data that goes unprocessed
 - Implementation
 - Asynchronous checkpointing (distributed snapshot)
 - Flink application continues to process data during the checkpointing process

Dataflow programming

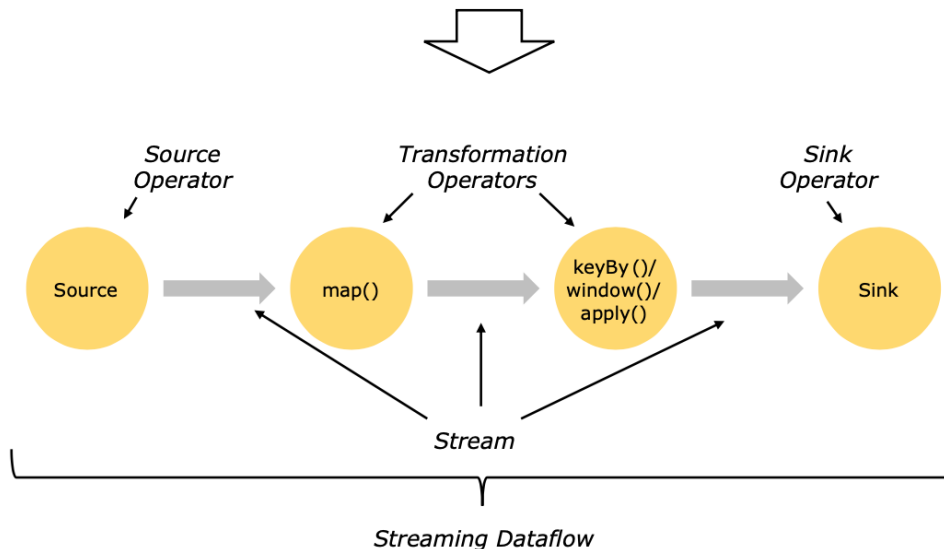
```
DataStream<String> lines = env.addSource (  
    new FlinkKafkaConsumer<> (...));  
  
DataStream<Event> events = lines.map((line) -> parse(line));  
  
DataStream<Statistics> stats = events  
    .keyBy("id")  
    .timeWindow(Time.seconds(10))  
    .apply(new MyWindowAggregationFunction());  
  
stats.addSink(new BucketingSink(path));
```

Source

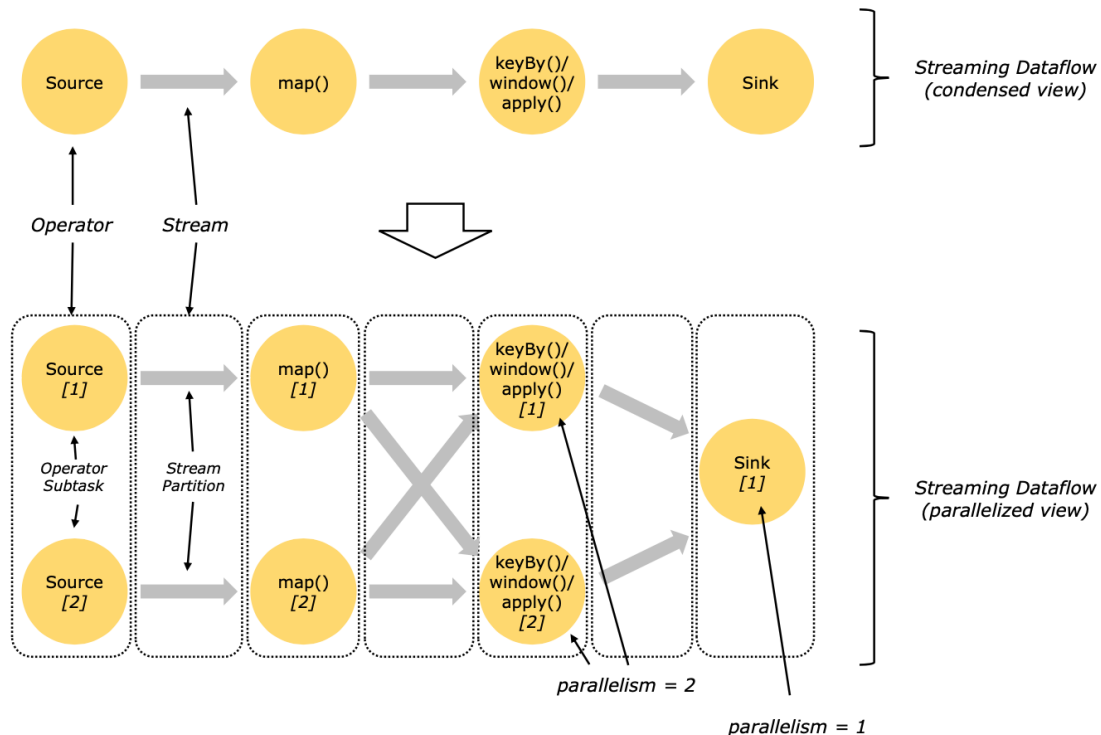
Transformation

Transformation

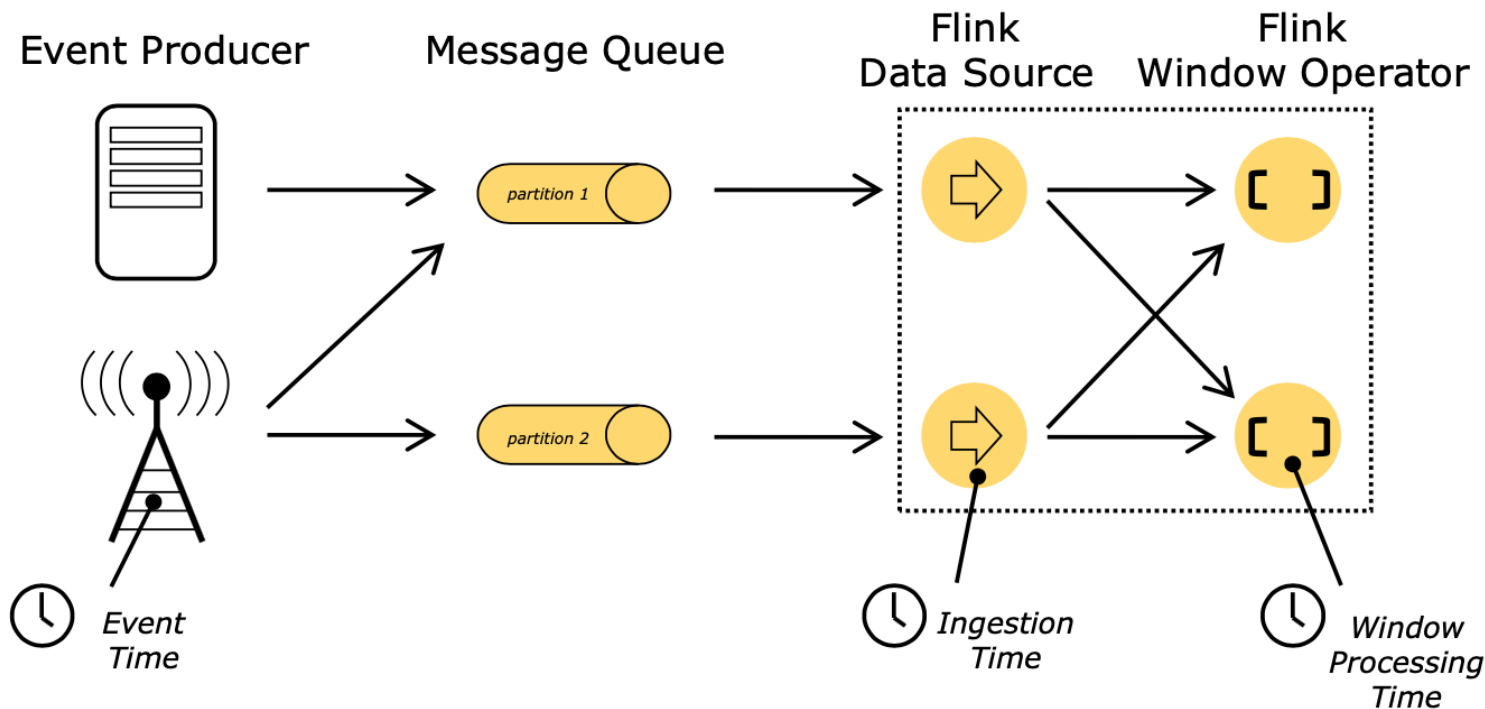
Sink



Physically: Parallel Dataflow

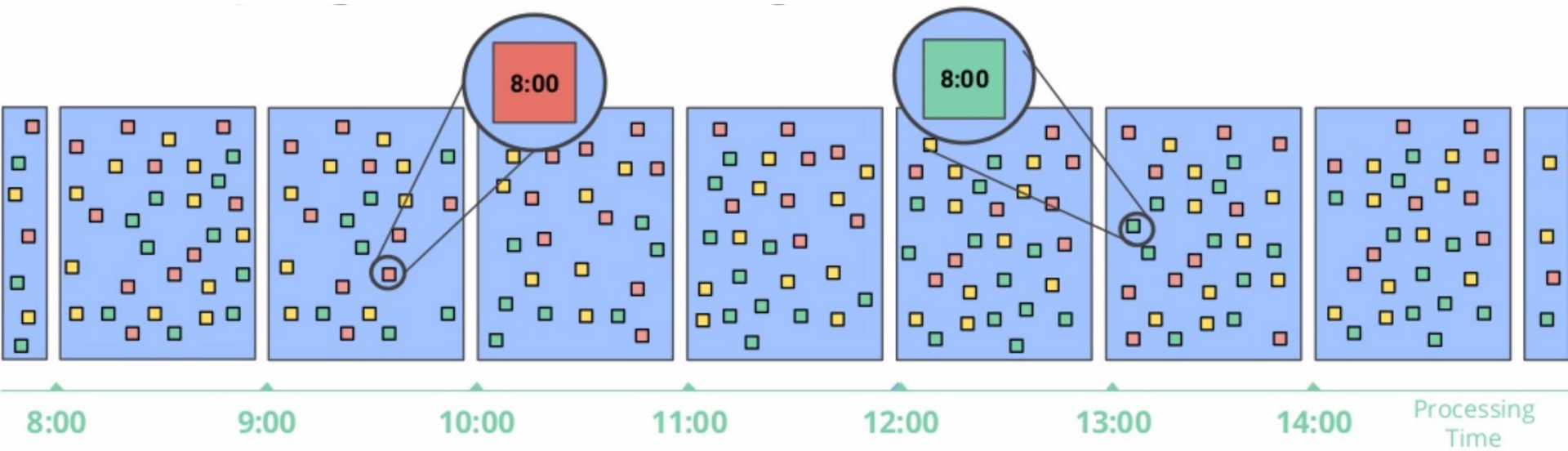


- **Event Time** is the time when an event was created. It is usually described by a timestamp in the events, for example attached by the producing sensor, or the producing service. Flink accesses event timestamps via [timestamp assigners](#).
- **Ingestion time** is the time when an event enters the Flink dataflow at the source operator.
- **Processing Time** is the local time at each operator that performs a time-based operation.

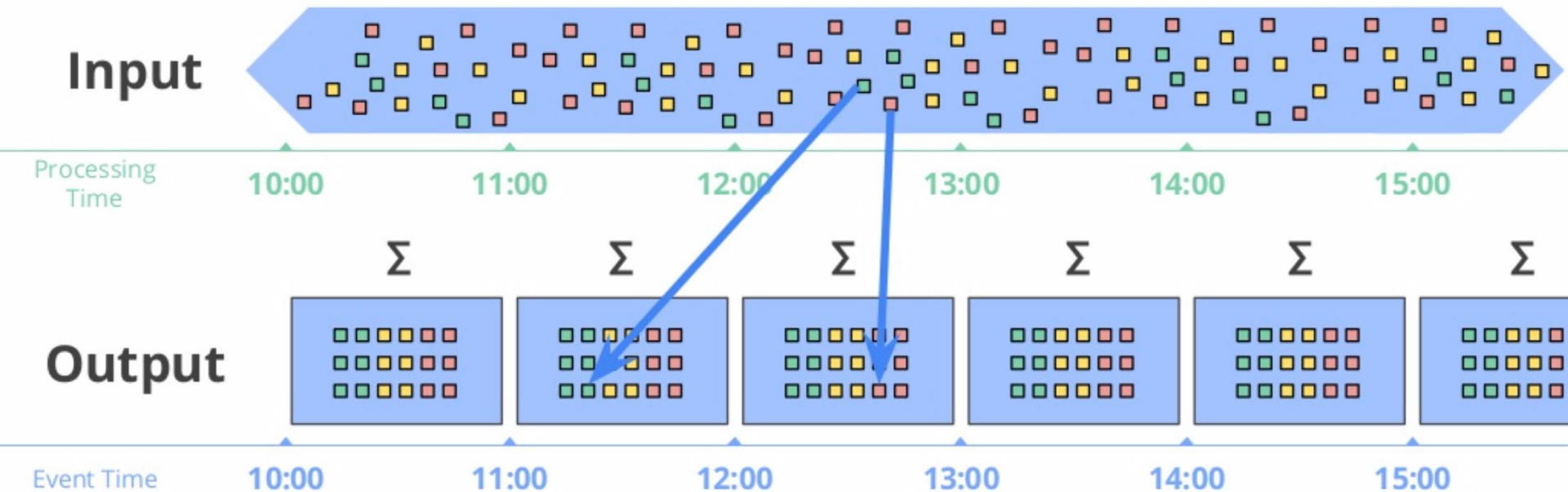


Dataflow programming model

- Originally:
 - **Processing-Time** Grouping / Window only
- Introduce windowing function to define
 - **Event-time** Data-oriented Grouping / Window

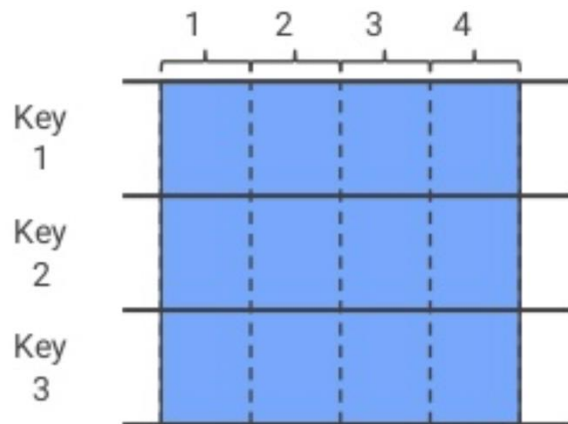


Grouping / Window by processing event-time

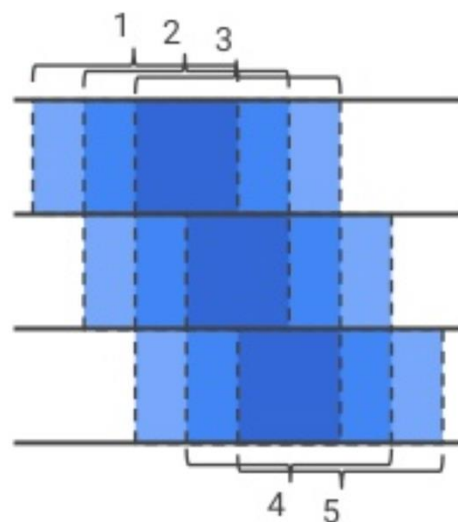


Windowing divides data into event-time-based finite chunks.

Fixed



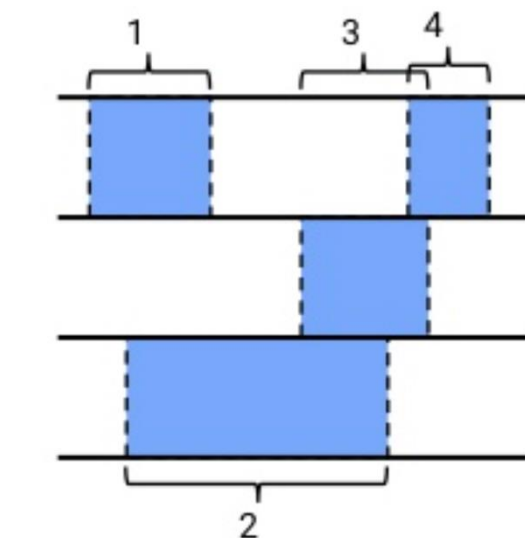
Sliding



A windowing function computes which window(s) an element belongs to. Temporal functions can be parameterized with **duration** and **frequency**.

Time

Sessions



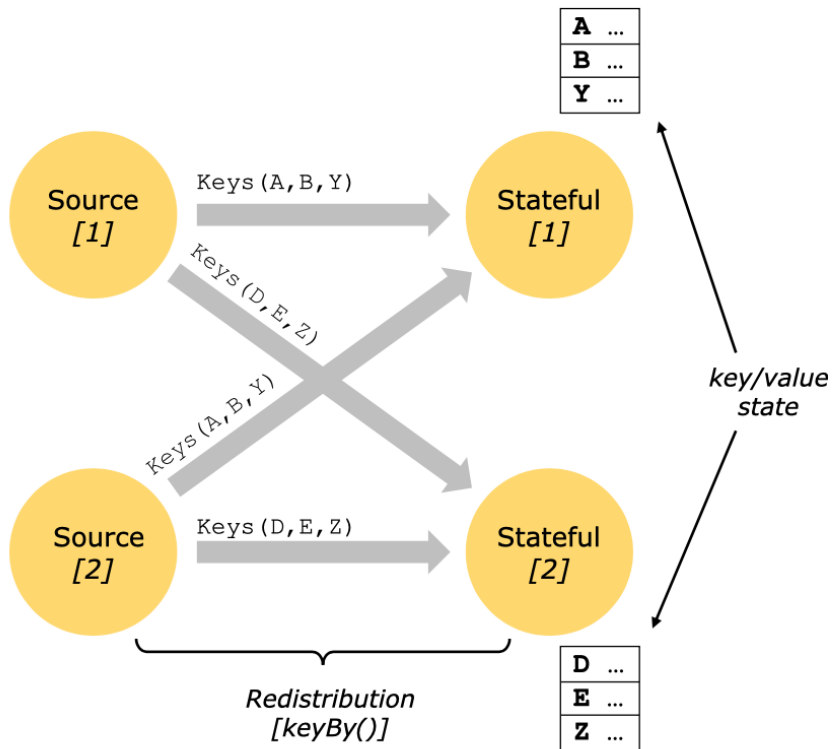
Unique per key - you can't know a priori when a session ends, so the windowing function is now also parameterized by state.



Time

Some operators are stateful

- E.g., window
 - SUM of all events



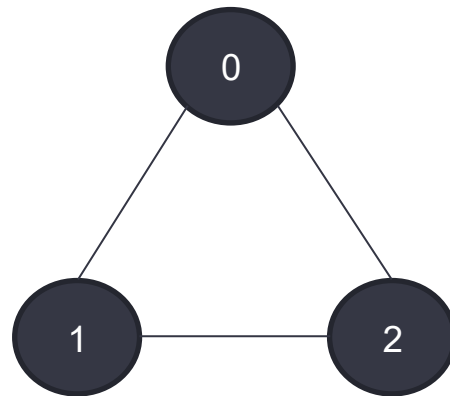
Exactly-once consistency requires distributed snapshot

Checkpointing

The central part of Flink's fault tolerance mechanism is drawing consistent snapshots of the distributed data stream and operator state. These snapshots act as consistent checkpoints to which the system can fall back in case of a failure. Flink's mechanism for drawing these snapshots is described in "[Lightweight Asynchronous Snapshots for Distributed Dataflows](#)". It is inspired by the standard [Chandy-Lamport algorithm](#) for distributed snapshots and is specifically tailored to Flink's execution model.

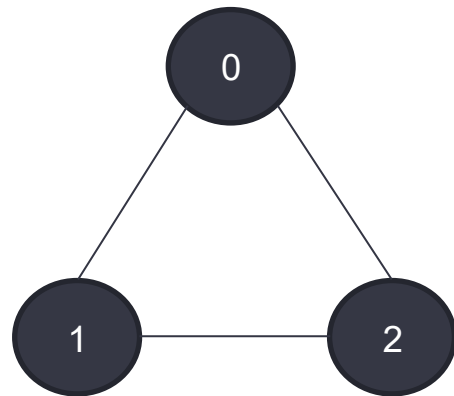
Snapshot: the state of the system

- Ex: token counting in 3 processes
 - An unknown, N , # of tokens circulating among them
 - Take a snapshot to know N at a particular moment
 - Tokens in-transit between two processes also count



Snapshot: the state of the system

- One triggers the snapshot process
 - send request messages to nodes to collect their # of tokens
- Assume process 0 takes the snapshot
 - Process 0: $N_{p0}=1$
 - Process 1 replies $N_{p1}=4$
 - Process 2 replies $N_{p2}=5$
 - $N = N_{p0} + N_{p1} + N_{p2} = 10$?
- Problems
 - What if there is a token in-transit from P1 to P2?



What is a Consistent Snapshot?

- Defined based on the notion of **cuts** and **consistent cuts**.
- A cut is a set of events, at least 1 event from each process

- Ex: {c,d,f,g,h} is a cut
- Ex: {a,b,c,d,g} is not

- A **consistent cut** is for each event it contains

- the events that casually ordered before it
- Let x and y are 2 events
 - If $y < x$ and x is in a consistent cut C
 - Then y is also in C

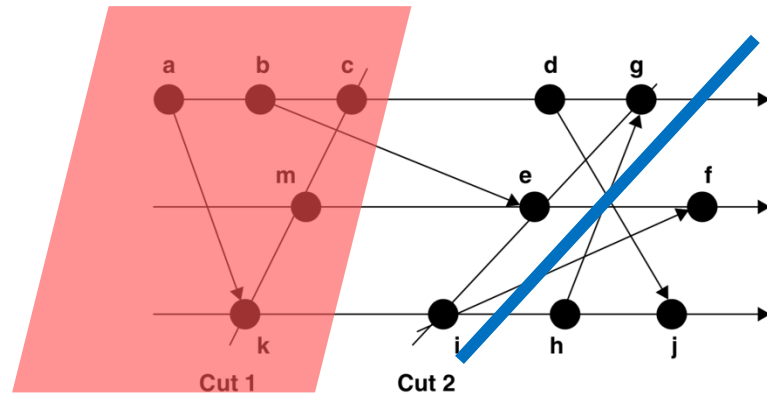
- Ex: Cut1 = {a,b,c,m,k} is a **consistent cut**

- Ex: Cut2 = {a,b,c,d,g,m,e,i} is not

- Because the event that cause g, which is h, is not in cut2

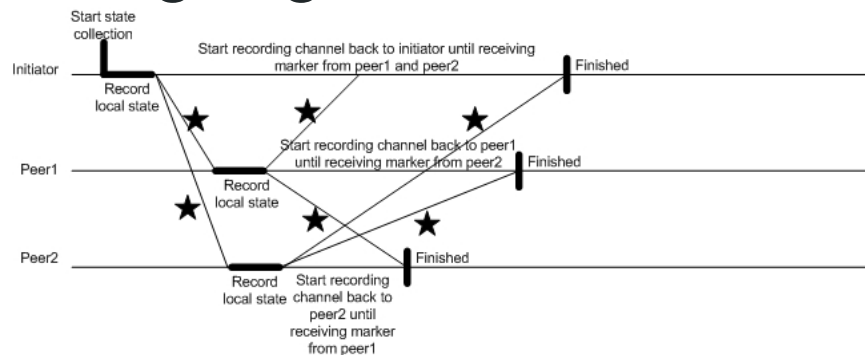
- As the system makes progress, the consistent cut is updated by new incoming events

- The frontier of a **consistent cut** is a **snapshot**
 - Ex: {c,m,k}
- An older consistent cut is a subset of a more recent consistent cut



Chandy-Lamport Snapshot Taking Algorithm

- Assume FIFO channel (e.g., TCP)
- Initiator i:
 - Record its own state
 - Send a marker * along all its outgoing channels c
 - Start (new) recording each incoming channel c_{yi} from y
- All:
 - On receiving a marker * from a channel c from source s
 - **If** 1st time seeing a marker
 - Record its own state
 - Pass the marker * to all its outgoing channels
 - Start (new) recording each incoming channel except the one from s
 - **Else**
 - Stop recording channel from s



Remember, the channels have no computational / storage power

- It is the job of the process to **record the state of the channels**

of messages sent: $N(N-1)$

Chandy-Lamport Snapshot Taking Algorithm

- Snapshot taking finish when:
 - Every process
 - Has recorded its local state and all incoming channels have stopped recording
- Next time, the initiator triggers snapshot by sending out a new marker **
- Good video:
 - <https://youtu.be/x1BCZ351dJk?t=2299>

